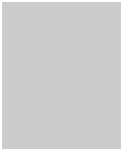


BooN: Boolean Network Analysis Software

Journal:	<i>Bioinformatics</i>
Manuscript ID	Draft
Manuscript type:	Applications Note
Date Submitted by the Author:	n/a
Complete List of Authors:	Delaplace, Franck; Paris-Saclay University, Computer Science; University of Evry-Val-d'Essonne, Computer Science
Portal Keywords:	Biological networks, Computational biology, Mathematical modeling, Software, Systems biology
Keywords:	Boolean Network, Controllability, Graph theory, Mathematical modeling, Systems biology, Biological networks



BooN: Boolean Network Analysis Software

Franck Delaplace^{1,*}

¹IBISC, Paris-Saclay University, Evry, France

*Corresponding author. franck.delaplace@univ-evry.fr

FOR PUBLISHER ONLY Received on Date Month Year; revised on Date Month Year; accepted on Date Month Year

Abstract

In this paper, we present an open source software, called **BooN**, which has been developed for the purpose of Boolean network analysis in Python. The distribution includes two main packages: one comprising a set of methods applied to the Boolean network object and the other for the graphical interface. The software encompasses all the requisite features for Boolean network analysis, including graphical network design, interaction graph drawing, model of dynamics, symbolic computation of stable states, and controllability analysis. **BooN** can be freely accessed on GitHub [1].

Key words: Boolean Network, Controllability, Software

1. Introduction

Boolean Network Analysis applied to biological systems studies was pioneered by Stuart Kaufman [2, 3] and René Thomas [4] in 70s. The objective was to develop a formal modeling framework for studying the dynamics of genetic networks, considering the available data for their analysis.

Now such a modeling framework can be considered as a Gold standard for biological system study as evidenced by the significant number of publications on biological network/system modeling (Figure 1). Indeed, these publications have mostly supplanted other modeling frameworks regarding quantity.

One of the most significant challenges for this modeling framework concerns the development of software platforms facilitating their use by a large number of users with a diverse range of skill sets. Such platforms should offer a comprehensive range of functionality and a user-friendly graphical interface.

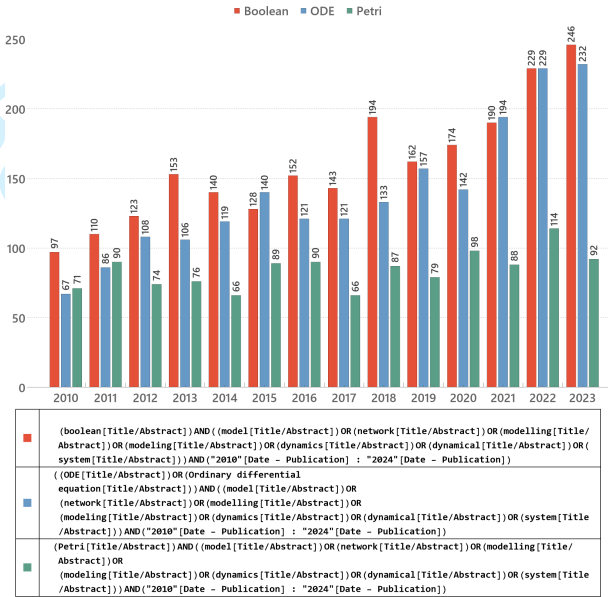
This article introduces **BooN** a Python-based platform that provides a comprehensive set of functionalities for Boolean network analysis and a user-friendly graphical interface for Boolean network manipulation.

Section 2 recalls the main features of a Boolean network analysis including equilibria computation and controllability analysis. Section 3 details the different functionalities of **BooN**. In conclusion (Section 4) we compare **BooN** to the main software distributions related to Boolean network analysis.

2. Boolean Network

2.1. Basic definitions

A *Boolean network* is a discrete dynamical system operating on Boolean variables X . It is defined as a system of Boolean equations of the form: $x_i = f_i(x_1, \dots, x_n), 1 \leq i \leq n$ where each f_i is a



3 queries were performed on PUBMED to count the number of publications concerning Boolean, ordinary differential equation, and Petri Net model related to Biological network or system studies. The used queries are described below the bar-chart.

Fig. 1: Number of publications per modeling framework on PUBMED.

logical propositional formula. We present in Figure 2 the dynamics

of the following Boolean Network:

$$F = \begin{cases} x_1 = (x_1 \vee x_2) \\ x_2 = (\neg x_3 \wedge x_1) \\ x_3 = (x_2 \wedge x_1) \end{cases} \quad (1)$$

A Boolean *state* of s is an interpretation of the variables in Boolean (ie., $s : X \rightarrow \mathbb{B}, \mathbb{B} = \{0, 1\}$). will denote the set of all states for variables belonging to X .

The analysis of a Boolean network is primarily focused on the dynamics to investigate the behavior of the dynamical system. In contrast to simulations performed with other models, such as ODE, the analysis of trajectories is achieved exhaustively for all trajectories, thereby ensuring a comprehensive and relevant study of the dynamics. Formally, a *model of dynamics* is defined by a labeled transition system where for each transition the states of agents are updated concerning an updating policy defined by a *mode*. Usually, two modes are used: the *asynchronous* mode where one variable exactly is updated per transition and the *synchronous* mode where all the variables are updated per transition.

As example, the *asynchronous* mode leads to the following labelled transition system $\langle \rightarrow, X, \mathbb{B}^n \rangle$ where the transition relation $\rightarrow \subseteq S_X \times X \times S_X$ is labelled by the updated agent, $\xrightarrow{x_i}$ such that:

$$s_1 \xrightarrow{x_i} s_2 \stackrel{\text{def}}{=} s_1 \neq s_2 \wedge s_2(x_i) = f_i(s_1) \wedge \forall x_j \in X \setminus \{x_i\} : s_2(x_j) = s_1(x_j).$$

A key objective of the analysis is to assess the equilibrium of the modeled network to gain insight into its long-term behavior. Basically, a state s is an *equilibrium* for $\rightarrow$, if it can be infinitely reached once met, ie., $\forall s' \in S_X : s \rightarrow^* s' \implies s' \rightarrow^* s$. The aim is to identify and characterize the attractors, which are defined as sets of mutually reachable equilibria. The attractors reduced to a single state are called *stable states* and can be efficiently and fastly computed by symbolic method while the computation of the other attractors (cyclic) remains exponential in general, limiting their investigation to small networks.

Finally, an *interaction graph* $\langle X, \rightarrow \rangle$ portrays the causal interactions between variables of a Boolean network. An interaction $x_i \rightarrow x_j$ exists if and only if x_i occurs as literal in a minimal DNF form of f_j , ie., $x_i \rightarrow x_j \stackrel{\text{def}}{=} x_i \in V(\text{DNF}(f_j))$. The interactions are refined by a sign stipulating the nature of the interaction: positive (+) or negative (-). Figure 2 illustrates the asynchronous dynamics of Boolean network (1) leading to a cyclic attractor (red) and a stable state (yellow), and its signed interaction graph.

2.2. Controllability

Recently, a significant body of research [5, 6, 7, 8, 9, 10, 11, 12] has been devoted to the study of controllability in Boolean networks. The objective of this research is to identify the necessary modifications to the equations governing the network to ascertain the means of reaching a given equilibrium state. Adding control extends the Boolean network to *Boolean control network* (BCN) where the *control parameters* u_i are added to the equation for figuring the control action. The classical modifications of equations consist in freezing a variable by setting it to True/1 or False/0. Basically, adding control extends the formula to include the control

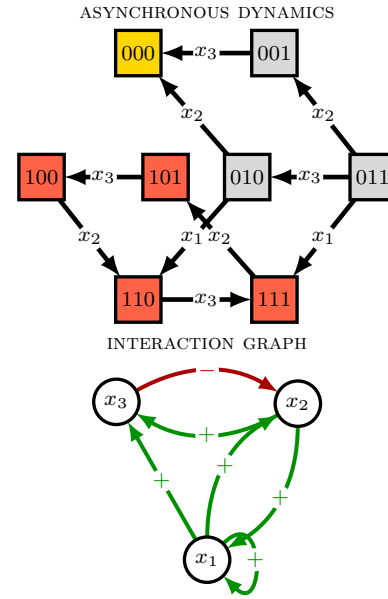


Fig. 2: Asynchronous dynamics and equilibria (in different colors for each attractor), interaction graph

parameters. A freeze occurs when the control parameters freeze ($u_i = 0$) otherwise it is inactive ($u_i = 1$). The formalization of the control extension can be achieved as follows for each equation:

$$\begin{aligned} x_i &= f_i(x_1, \dots, x_n) \wedge u_i^0 && \text{for freezing to 0} \\ x_i &= f_i(x_1, \dots, x_n) \vee \neg u_i^1 && \text{for freezing to 1.} \end{aligned}$$

Note that when $u_i^0 = 0$ then the equation becomes $x_i = 0$ and similarly when $u_i^1 = 0$, $x_i = 1$. The Boolean network (1), subjected to control thus becomes :

$$\begin{cases} x_1 = (x_1 \vee x_2) \wedge u_1^0 \vee \neg u_1^1 \\ x_2 = (\neg x_3 \wedge x_1) \wedge u_2^0 \vee \neg u_2^1 \\ x_3 = (x_2 \wedge x_1) \wedge u_3^0 \vee \neg u_3^1 \end{cases} \quad (2)$$

The BCN has now 6 control parameters. With this method, the control is applied on nodes because the variable is set to a Boolean value when the control freezes. The control can be also more selectively applied on links [7]. As the Boolean network is modified when the control freezes, the dynamics is altered leading to different equilibria. To illustrate this situation, if the objective is to reach $x_3 = 1$ at a stable state, the sole control that will achieve this outcome is to activate u_2^1 ie., $u_2^1 = 0$ while $u_i^c = 1$ for the other configurations, the resulting BCN and its dynamics are shown in Figure 3. Now only one new stable state exists (111).

The primary challenge is to accurately determine the control parameters' freeze actions to regulate the dynamics in order to achieve the desired equilibrium. As previously referenced, a variety of methods have been developed to address this issue.

Controllability analysis is a valuable investigative tool in drug target discovery and disease etiology. Typically, a biomarker profile identifies a specific state, either diseased or healthy state. From control analysis, we can determine which variables should be turned ON or OFF to meet the desired binary biomarker profile at a stable state. This allows us to properly identify the actions that should be applied to genes, either over-expression or inhibition.

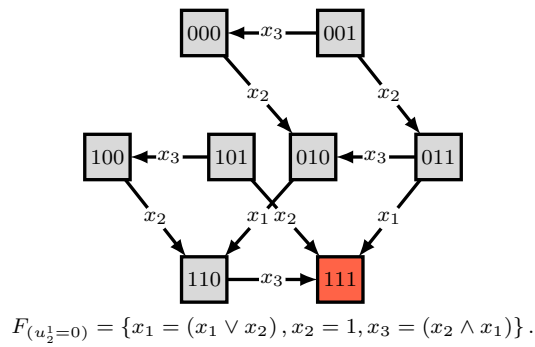


Fig. 3: Controlled network of F by assigning $u_2^1 = 0$.

3. BooN

BooN facilitates the analysis of the Boolean network including the network design, the equilibrium computation and the controllability analysis. It is freely available in open source under Gith Hub [1]. **BooN** has also been added in Colomoto [13] repository collecting software related to Biological network analysis. The distribution includes the sources, a complete documentation of the different methods applied to **BooN** object, and various examples including a real application of the controllability analysis on Breast cancer leading to drug target prediction. **BooN** was developed in Python and composed of 3 packages:

- **boon** is the core library providing a comprehensive set of methods for manipulating the **BooN** object.
- **logic** is a sub-library of **boon** containing the basic functions related to propositional logic: basic functions, formula pretty printing, CNF conversion, and prime implicants computation.
- **boonify** is the user-friendly graphical interface (GUI) interfacing the **boon** function based on PyQT5.

BooN has an internal file format for the storage of Boolean networks. However, for the purpose of interoperability with other applications, Boolean networks in BoolNet, SBML, or Python-based format can also be imported.

Figure 4 illustrates the full range of analysis features accessible via different windows using the GUI for Boolean network (1).

Concerning the GUI, the design of the Boolean network is completely achieved from the drawing of the interaction graph and can be changed from the direct modification of the formula in the Formulas View window (Figure 4). Moreover, the different analysis (Dynamics w.r.t. a mode, Controllability, symbolic Stable state computation) are accessible via a menu and buttons located in the both sides of the main window where the interaction graph is drawn. Comprehensive online help is also available.

More technically, dynamics refers to calculating transitions between states in a mode that defines the state updating policy. The symbolic computation of stable states efficiently enumerate the interpretations of the following equivalence stating the fixed point of the dynamics:

$$\bigwedge_i f_i(x) \iff x_i.$$

The resolution is achieved by a SAT solver, specifically the z3-Solver [14]. Finally, the controllability analysis refers to the prime implicants computation as described in [7]. These prime implicants are used to extract the negative control parameters ($u_i^c = 0$) for

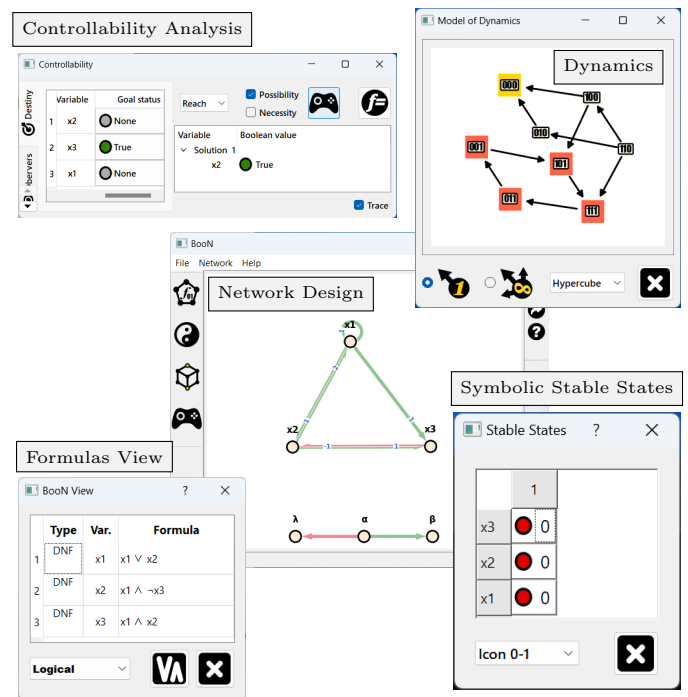


Fig. 4: Overview of the **BooN** functionalities/windows using GUI.

defining the *core* which is the minimal set of active control parameters under the inclusion. The prime implicant algorithm that has been implemented is based on the translation of the problem into an integer linear programming problem [15] and the used integer linear programming solver is Pulp [16].

4. Conclusion

Several software packages are also available for Boolean network analysis. These are written in different languages and offer a range of functions for performing this analysis.

GINSIM [17] provides an interactive design of the Boolean Network, including annotations using a GUI. It offers a stepwise simulation and full dynamics for any mode, as well as a computation of stable states based on symbolic analysis and a full description of attractors from a brute force algorithm limiting its application to small network. It has been developed in JAVA.

BoolNet is a R-package also devoted to Boolean network analysis. It tackles with the synchronous and asynchronous analysis and also probabilistic Boolean network. It includes functions for various simulations, attractor search, and robustness analysis but does not contain symbolic algorithm for stable states computation. The specification of a Boolean network is achieved from formula description. BoolNet is reduced to a package and does not contain a graphical interface for facilitating the network design.

BoolSi [18] is an open-source cross-platform command line tool restricted to simulations of synchronous Boolean networks. It supports parallel processing using MPI. BoolSi is used to model the behavior of complex dynamic networks and allows for identification and statistical analysis of network attractors.

BooN is similar to GINSIM in terms of its functionalities. In comparison to the BoolNet package, Boolean Networks is devoted to pure Boolean networks. However, it provides a package and a

graphical interface which provides a convenient way to visualize and interact with the network. It also offers unique capabilities for controllability analysis which is not available in other software.

A number of software applications are devoted to controllability analysis, as CABEAN [19] or Bonesis [20]. However, they are focused on this specific analysis and do not encompass the other aspects related to Boolean dynamics analysis addressed by the aforementioned software.

In conclusion, **BooN** is a software solution tailored for Boolean network dynamics analysis, featuring a unique ability to conduct controllability analysis. Furthermore, its graphical interface allows for the straightforward manipulation of Boolean networks.

References

1. Franck Delaplace. Franck-Delaplace/BooN, September 2024.
2. Stuart Kauffman. Homeostasis and differentiation in random genetic control networks. *Nature*, 224:177–178, 1969.
3. Leon Glass and Stuart A Kauffman. The logical analysis of continuous, non-linear biochemical control networks. *Journal of Theoretical Biology*, 39(1):103–129, 1973.
4. René Thomas. Boolean formalization of genetic control circuits. *Journal of Theoretical Biology*, 42(3):563–585, 1973.
5. Laura Cifuentes-Fontanals, Elisa Tonello, and Heike Siebert. Control in Boolean Networks With Model Checking. *Frontiers in Applied Mathematics and Statistics*, 8, April 2022.
6. Jérémie Pardo, Sergiu Ivanov, and Franck Delaplace. Sequential reprogramming of biological network fate. *Theoretical Computer Science*, 872:97–116, June 2021.
7. Célia Biane and Franck Delaplace. Causal Reasoning on Boolean Control Networks Based on Abduction: Theory and Application to Cancer Drug Discovery. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 16(5):1574–1585, September 2019.
8. Hugues Mandon, Cui Su, Stefan Haar, Jun Pang, and Loïc Paulevé. Sequential Reprogramming of Boolean Networks Made Practical. In Luca Bortolussi and Guido Sanguinetti, editors, *Computational Methods in Systems Biology*, pages 3–19, Cham, 2019. Springer International Publishing.
9. David Murrugarra, Alan Veliz-Cuba, Boris Aguilar, and Reinhard Laubenbacher. Identification of control targets in Boolean molecular network models via computational algebra. *BMC Systems Biology*, 10(1):94, September 2016.
10. Jorge G. T. Zañudo and Réka Albert. Cell Fate Reprogramming by Control of Intracellular Network Dynamics. *PLOS Computational Biology*, 11(4):e1004193, April 2015.
11. Pey-Chang Kent Lin and Sunil P. Khatri. Application of Max-SAT-based ATPG to optimal cancer therapy design. *BMC Genomics*, 13(6):S5, October 2012.
12. Ritwik Layek, Aniruddha Datta, Michael Bittner, and Edward R Dougherty. Cancer therapy design based on pathway logic. *Bioinformatics*, 27(4):548–555, 2011.
13. Aurélien Naldi, Céline Hernandez, Nicolas Levy, Gautier Stoll, Pedro T. Monteiro, Claudine Chaouiya, Tomáš Helikar, Andrei Zinovyev, Laurence Calzone, Sarah Cohen-Boulakia, Denis Thieffry, and Loïc Paulevé. The CoLoMoTo Interactive Notebook: Accessible and Reproducible Computational Analyses for Qualitative Biological Networks. *Frontiers in Physiology*, 9, June 2018.
14. Leonardo de Moura and Nikolaj Bjørner. Z3: An Efficient SMT Solver. In C. R. Ramakrishnan and Jakob Rehof, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340, Berlin, Heidelberg, 2008. Springer.
15. C. Pizzuti. Computing prime implicants by integer programming. In *Proceedings Eighth IEEE International Conference on Tools with Artificial Intelligence*, pages 332–336, 1996.
16. Stuart Mitchell, Michael O’Sullivan, and Iain Dunning. PuLP: A Linear Programming Toolkit for Python.
17. A. Naldi, D. Berenguier, A. Fauré, F. Lopez, D. Thieffry, and C. Chaouiya. Logical modelling of regulatory networks with GINsim 2.3. *Bio Systems*, 97(2):134–139, August 2009.
18. Vladyslav Oles and Anton Kukushkin. BoolSi: A Tool for Distributed Simulations and Analysis of Boolean Networks. *Journal of Open Research Software*, 8(1), October 2020.
19. Cui Su and Jun Pang. CABEAN: A software for the control of asynchronous Boolean networks. *Bioinformatics*, 37(6):879–881, March 2021.
20. Stéphanie Chevalier, Déborah Boyenval, Gustavo Magaña-López, Théo Roncalli, Athénaïs Vaginay, and Loïc Paulevé. BoNesis: A Python-Based Declarative Environment for the Verification, Reprogramming, and Synthesis of Most Permissive Boolean Networks. In Roberta Gori, Paolo Milazzo, and Mirco Tribastone, editors, *Computational Methods in Systems Biology*, pages 71–79, Cham, 2024. Springer Nature Switzerland.